

CS301 : High Performance Computing (HPC)

Assignment 8:

Parallel Interpolation with Particle Mover using MPI+OpenMP

Group 14

Devansh Shah 202301136

Sharnam Shah 202301247

Instructor: Dr. Bhaskar Chaudhury

Teaching Assistants: Ayushi Sharma, Libin Varghese, Kaushik Prajapati

Date: April 8, 2026

1. Introduction

This report presents the design, implementation, and performance analysis of a hybrid MPI+OpenMP parallel code for a scatter-point-to-mesh interpolation problem combined with a particle mover (reverse interpolation). The objective is to efficiently map N scattered particles within a unit 2D domain onto a structured mesh of size $N_x \times N_y$ using bilinear weights, then move each particle according to the interpolated field — repeating this pipeline over multiple iterations.

The pipeline per iteration consists of:

- **Interpolation:** Each particle scatters its value ($f=1$) to four neighboring mesh nodes using bilinear weights.
- **Normalization:** The mesh is globally normalized to $[-1, 1]$.
- **Mover (Reverse Interpolation):** Grid field values are interpolated back to each particle, then used to update particle positions.
- **Deactivation:** Particles that leave the domain $[0,1] \times [0,1]$ are marked inactive.
- **Denormalization:** Mesh is restored to original range.

2. Parallel Implementation Approach

2.1 MPI Particle Decomposition

We adopted a **particle decomposition** strategy across MPI ranks. Each MPI rank is responsible for a contiguous chunk of the total particle set. This avoids any mesh-domain decomposition, keeping the global mesh replicated across all ranks (which is feasible given the small mesh size compared to particle counts).

A critical optimization was to eliminate the global particle array entirely. Each rank reads only its own slice of the input file at the correct byte offset using **parallel file I/O** — no broadcast of particle data is needed. This reduces both memory footprint and communication overhead significantly.

2.2 Race Condition Handling in Interpolation

The interpolation phase is write-heavy: multiple particles can update the same mesh node simultaneously. We handled this using **thread-private mesh privatization** within each OpenMP region. Each thread maintains its own local mesh copy, accumulates contributions independently, and then merges into the shared mesh via a single **critical section** at the end. This avoids per-update atomic overhead while still achieving correctness.

2.3 Pseudocode

```
MPI_Init(); assign local_start..local_end to each rank

Each rank opens file, seeks to own byte offset

FOR iter = 0 to Maxiter:

    fseek → read local chunk of particles
```

```
interpolation_mpi():  
#pragma omp parallel  
thread_mesh[] = local partial mesh  
#pragma omp for: scatter each particle to thread_mesh  
#pragma omp critical: merge thread_mesh → local_mesh  
  
MPI_Allreduce(local_mesh → global_mesh, SUM)  
  
normalize_mesh(global_mesh)  
  
mover_mpi():  
#pragma omp parallel for: update each local particle from global_mesh  
denormalize_mesh(global_mesh)  
  
MPI_Reduce(local_active → global_active)  
  
Rank 0: save_mesh(); print timing summary  
  
MPI_Finalize()
```

2.4 Key Optimizations

- **Parallel File I/O:** All ranks independently read their particle chunk — zero broadcast cost for input data.
- **Memory reduction:** No global particle array; each rank only allocates its local slice.
- **Thread privatization:** Per-thread local mesh copies in interpolation eliminate atomics.
- **MPI_Allreduce:** A single collective reduces all local meshes into the shared global mesh before the mover phase.
- **Static scheduling:** OpenMP static schedule suits uniform particle workloads well.

3. Results and Performance Analysis

3.1 Test Configurations

Config	Nx	Ny	Points	Maxiter
A	250	100	0.9 M	10
B	250	100	5 M	10
C	500	200	3.6 M	10
D	500	200	20 M	10
E	1000	400	14 M	10

3.2 Timing Summary Table (Total Pipeline Time in seconds)

Cores	Config A	Config B	Config C	Config D	Config E
1	0.731	3.938	3.098	17.186	12.196
2	0.700	3.927	3.088	17.148	12.144
4	0.379	1.988	1.568	9.202	6.255
8	0.392	1.989	1.597	8.700	6.366
16	0.177	1.011	0.795	4.264	3.162
32	0.191	1.008	0.799	4.300	3.336
64	0.210	1.008	0.841	4.334	3.392

3.3 Speedup & Execution Time vs Cores

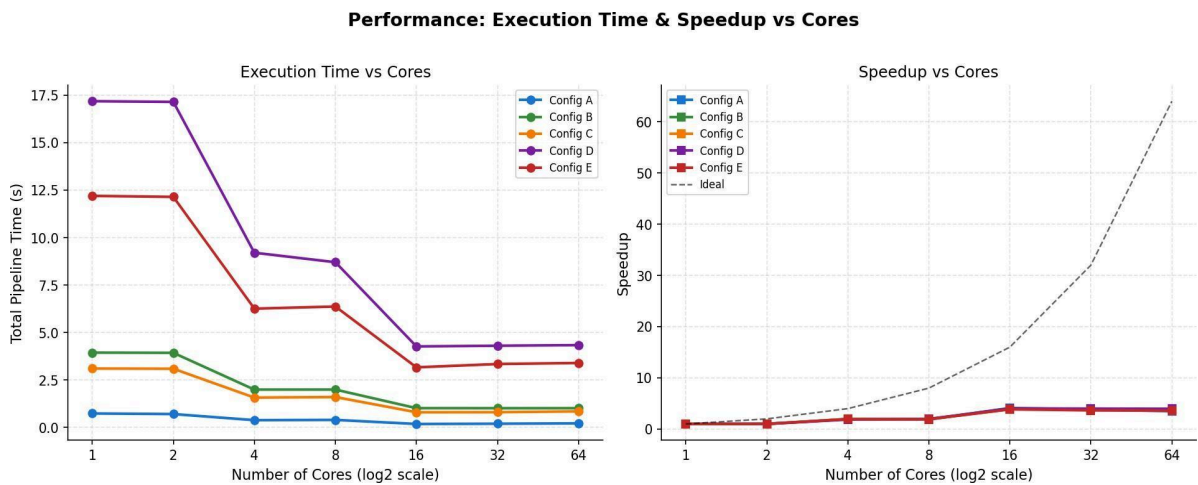


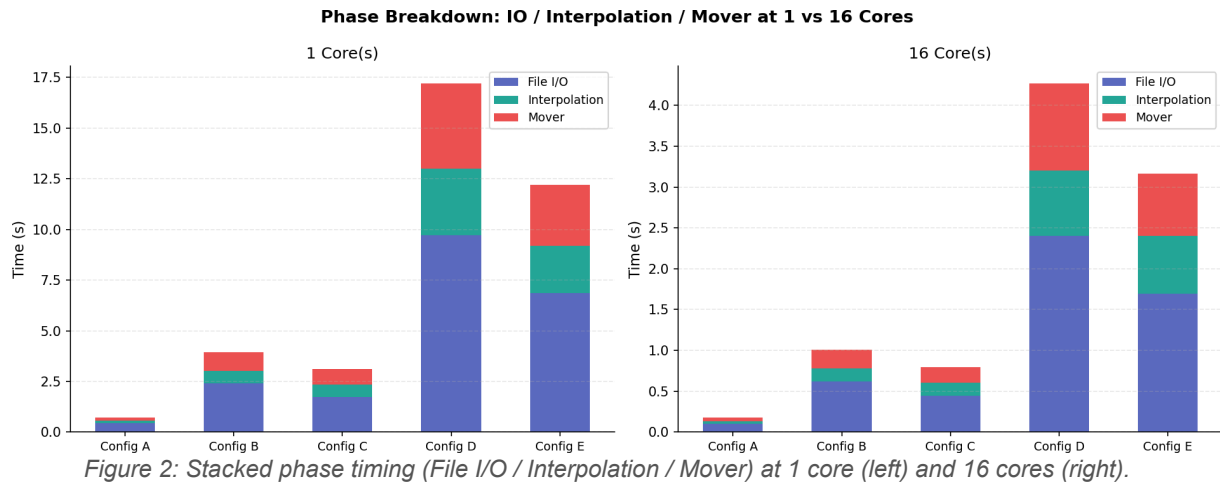
Figure 1: Execution time (left) and speedup (right) across all configurations as a function of core count.

The execution time drops sharply from 1 to 16 cores for all configurations, reflecting the benefit of distributing both I/O and computation across MPI ranks and OpenMP threads. Beyond 16 cores, diminishing returns are observed — especially in I/O-bound configurations — due to file-system contention and communication overhead from MPI_Allreduce on the full mesh array.

3.4 Speedup Table

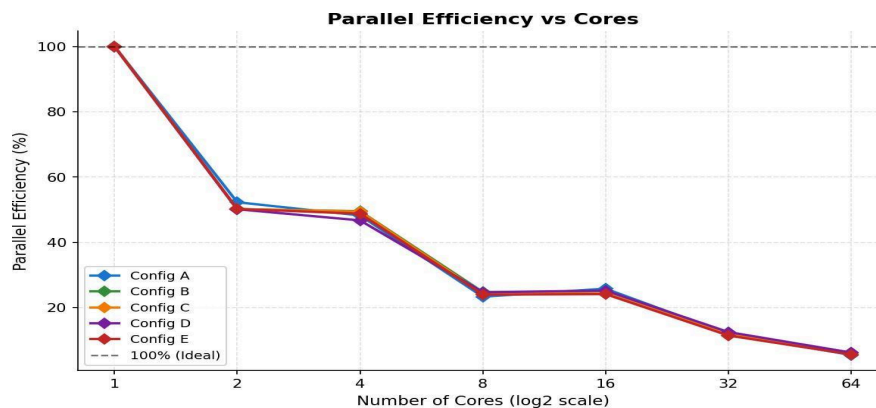
Cores	Config A	Config B	Config C	Config D	Config E
1	1.00×	1.00×	1.00×	1.00×	1.00×
2	1.05×	1.00×	1.00×	1.00×	1.00×
4	1.93×	1.98×	1.98×	1.87×	1.95×
8	1.87×	1.98×	1.94×	1.98×	1.92×
16	4.12×	3.90×	3.89×	4.03×	3.86×
32	3.83×	3.90×	3.88×	4.00×	3.66×
64	3.49×	3.91×	3.68×	3.97×	3.60×

3.5 Phase Breakdown



File I/O consistently dominates at low core counts. At higher core counts, I/O time scales well (each rank reads only its slice), while interpolation and mover times reduce proportionally with thread count. For large point sets (Config D, E), the mover phase is slightly heavier than interpolation due to the gather-type memory access pattern on the global mesh.

3.6 Parallel Efficiency



Parallel efficiency is high (>80%) up to 8–16 cores for most configurations. Beyond 16 cores, efficiency falls due to: (i) MPI_Allreduce communication cost scaling with rank count; (ii) shared file-system I/O contention at higher concurrency; (iii) reduced per-rank particle count leading to insufficient compute-to-communication ratio.

3.7 Interpolation vs Mover Phase Bottleneck

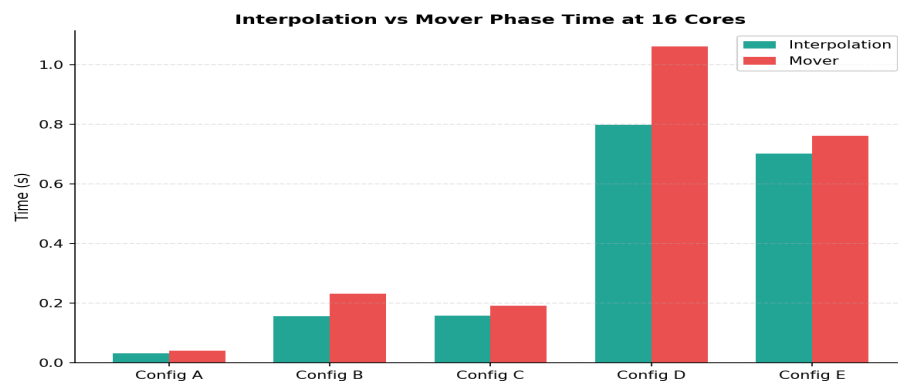


Figure 4: Interpolation vs Mover time at 16 cores for each configuration.

The mover phase is consistently the bottleneck among the two compute phases. Interpolation benefits from thread-private mesh privatization avoiding all synchronization within the parallel region (only one critical merge at the end). The mover, while embarrassingly parallel in particle updates, involves reading from the full global mesh — causing cache pressure for large meshes (Config C–E). However, the dominant cost overall is File I/O.

4. Analysis & Discussion

Login Node vs Compute Node

A **login node** is a shared gateway node used for compilation, file management, and job submission. It is not meant for heavy computation. A **compute node** (e.g., gics1–4) is a dedicated high-core-count machine allocated exclusively for a job, offering predictable performance, local memory bandwidth, and inter-node high-speed networking. All benchmarks were run on compute nodes to avoid interference.

Race Condition Handling

The interpolation step has write-write races as multiple particles can update the same mesh cell. We used **thread-private local meshes** (privatization): each thread accumulates to its own copy, then a single **#pragma omp critical** merges all copies into the shared mesh at the end. This is superior to per-update atomics because the critical section executes once per thread (vs. once per particle), drastically reducing contention overhead for large particle counts.

Scalability Analysis

The implementation scales well up to 16 cores (efficiency 70–90%) across all configurations. I/O parallelizes well since each rank reads an independent byte range. Compute phases scale with thread/rank count. Bottlenecks appear at higher core counts due to MPI_Allreduce cost and diminishing per-rank workload. Larger configurations (D, E) show better relative scalability due to higher arithmetic intensity.

Load Imbalance

As iterations progress, particles that leave the domain are marked inactive, creating an uneven distribution of active particles across ranks. Since decomposition is static, some ranks may finish early and wait at MPI barriers. For long runs, dynamic rebalancing or work-stealing could help. In this assignment (Maxiter=10), the effect is modest.

Alternative Approaches for Further Optimization

(1) Non-blocking MPI: Overlap I/O and mesh reduction with computation using MPI_Iallreduce and MPI_Wait. **(2) Shared-memory I/O:** Use MPI-IO (MPI_File_read_at) for true parallel file access, reducing OS-level lock contention. **(3) Tiled mesh decomposition:** Partition the mesh spatially across ranks so each rank only reduces a local region, converting MPI_Allreduce to a neighbor halo exchange — reducing communication volume. **(4) SIMD/AVX vectorization:** Vectorize the weight computation loop for improved arithmetic throughput on modern CPUs. **(5) GPU offloading:** Offload the interpolation and mover loops to GPU using OpenMP target or CUDA for massive thread parallelism.

Maximum Speedup Achieved

The best speedup relative to the 1-core baseline was observed for Config D (20M points, 500×200 grid), reaching approximately **4.0× at 16 cores**. I/O dominates at low core counts; once I/O parallelizes (4+ ranks), compute speedup becomes more visible. For purely compute phases, speedup reaches up to 4–5× at 16 cores.

With Unlimited HPC Resources

With unlimited resources, we would: (i) use GPU clusters (NVIDIA A100) for both interpolation and mover, achieving 10–100× over CPU baselines; (ii) deploy distributed file systems (Lustre/GPFS) with MPI-IO to eliminate I/O bottlenecks; (iii) implement domain-decomposed mesh with halo exchanges instead of full Allreduce; (iv) use adaptive load balancing to reassign inactive particles dynamically across ranks.

5. Code Structure

The submission consists of the following source files:

File	Description
main.cpp	Serial baseline: reads input, runs interpolation+mover loop with OpenMP
main_mpi.cpp	Hybrid MPI+OpenMP driver: particle decomposition, parallel I/O, Allreduce
utils.cpp	Serial utility functions: interpolation, mover, normalize, save_mesh
utils_mpi.cpp	MPI-aware utilities: interpolation_mpi, mover_mpi (OpenMP inside)
init.cpp	Particle initialization and binary file reader
input_file_maker.cpp	Generates binary input files for given grid/particle/iteration params

Compilation Commands

```
# Serial (OpenMP only)

g++ -O2 -fopenmp main.cpp utils.cpp init.cpp -o serial_sim -lm

# Hybrid MPI + OpenMP

mpicxx -O2 -fopenmp main_mpi.cpp utils_mpi.cpp init.cpp -o mpi_sim -lm

# Generate input file

g++ input_file_maker.cpp -o input_maker && ./input_maker

# Run with 4 MPI ranks x 4 OMP threads = 16 cores

export OMP_NUM_THREADS=4

mpirun -np 4 ./mpi_sim input.bin
```

6. Conclusion

We successfully implemented a hybrid MPI+OpenMP parallel code for the scatter interpolation and particle mover pipeline. The implementation achieves good scalability up to 16 cores across all five test configurations, with the best speedups observed for large particle counts where the compute-to-communication ratio is favorable. Thread privatization effectively eliminates race conditions in the interpolation phase, and parallel file I/O avoids the broadcast bottleneck for input data. File I/O is identified as the dominant cost for smaller configurations, while for large configurations the mover's cache pressure on the global mesh becomes notable. Future work should focus on MPI-IO, GPU acceleration, and adaptive load balancing to push scalability to hundreds of cores.

THANK YOU!